

Stream B — §6.AK-Compliant Video-Issue Fix Spec: Device-Gate Coverage for Every Claimed Fix

Date: 2026-06-26 **Status:** Draft **Operator directive:** "No distribute unless every CHANGELOG-claimed user-visible fix has an EXECUTED+PASSED covering device Challenge for the exact shipped artifact." **Relationship to §6.AK:** This spec implements §6.AK clauses 1–7 as a mechanical per-cycle workflow. The incident that birthed the clause: 627a0d58 (1076 shipped C00-only; operator confirmed all video issues still broken post-distribute). Forensic anchor at `.lava-ci-evidence/sixth-law-incidents/2026-06-26-c00-only-gate-shipped-broken-flows.json`.

Classification: project-specific (the cycle-coverage gate pattern is reusable across any HelixConsumption project that ships device-tested artifacts; the specific issue IDs LVA-079..LVA-091 and CHANGELOG conventions are Lava-specific).

1. §6.AK Gate Requirements Summary

§6.AK (added 2026-06-26, commit 627a0d58) establishes a **coverage-intersection gate** that no artifact may be distributed unless every CHANGELOG-claimed user-visible fix and every operator-reported issue addressed in that cycle has an EXECUTED+PASSED covering device Challenge for the EXACT artifact being shipped. The seven clauses:

1.1 Coverage-Intersection Gate (clause 1)

`scripts/firebase-distribute.sh` MUST refuse unless the §6.Z evidence file enumerates, **per CHANGELOG-claimed user-visible fix**, the covering Challenge FQN + its device PASS row from the same-SHA gate run. A claimed fix with no executed+passed covering Challenge is a

distribute-blocker. The CHANGELOG's user-visible bullets are the claim-set; the gate's executed-PASS Challenge set MUST be a superset of the claim-set's covering Challenges.

1.2 Reproduce-First ON DEVICE for UI/flow issues (clause 2)

For any user-reported issue whose surface is a screen/flow (search, onboarding, provider selection, rendering, navigation), the reproduction test MUST be a **device Challenge that drives the same user journey**, not a JVM unit test. The failing-first RED run against the un-fixed build MUST be recorded before the fix; the GREEN run against the fixed build is the acceptance evidence.

1.3 Crash reproduction MUST match the obtained stack (clause 3)

Every crash fix MUST be preceded by a device Challenge that reproduces the **exact** Crashlytics stack trace (class + top frame) the crash reported. If the Crashlytics data cannot be obtained, the crash is NOT reproducible and is NOT shippable as "fixed".

1.4 No trusting inherited "fixed" claims (clause 4)

A fix claimed by a prior session/commit that is carried into a distribute cycle is treated as **UNVERIFIED** until re-confirmed by an executed device Challenge in THIS cycle. An unverified inherited claim MUST NOT appear in the CHANGELOG as a fix, and its workable item stays open until its covering Challenge runs GREEN on the gate.

1.5 Per-operator-video reproduction set (clause 5)

When the operator provides a manual-testing video, EVERY distinct issue in the vision-analysis MUST gain a covering device Challenge that drives that issue's exact journey, reproduce-first (RED on the build the video was recorded against, GREEN on the fixed build). The matching workable item cannot move to a closeable state until that RED-then-GREEN device Challenge exists.

1.6 The gate run is truth; the CHANGELOG is the claim (clause 6)

If the executed-PASS Challenge set does not cover every CHANGELOG claim, the honest action is to EITHER run the missing Challenges (and fix what fails) OR strike the unverified claims from the CHANGELOG and hold them open — NEVER distribute the gap.

1.7 Inheritance (clause 7)

Applies recursively to every submodule, every feature, every artifact. Submodule constitutions MAY add stricter coverage requirements but MUST NOT relax this clause.

2. Per-Video-Issue Coverage Map

All issues below are extracted from the workable items DB (docs/workable_items.db) for IDs LVA-079 through LVA-091, cross-referenced against the video Screen_Recording_20260626_122718 (recorded against 1076) and the operator's vision analysis.

The column "Covering Device Challenge FQN" names the Challenge that must be written or extended. "Reproduce-First Mutation" names the deliberate break and the expected failure message.

2.1 LVA-083 (Video #1) — Search returns ZERO results then 'Something went wrong'

Field	Value
ID	LVA-083
Title	Video #1 — Search returns ZERO results then 'Something went wrong' Error (primary function unusable)
Status	In progress
Priority	P0
Root cause	Multi-factorial. (a) 401 auth-header overwrite in AuthInterceptor (fixed in 1072 — shipped). (b) Engine response exceeds client readTimeout (30s) because the single-provider handler at search.go:64 has NO context.WithTimeout deadline; fixed in 1073 but 1073

Field	Value
	was never distributed (1075 branched from it but was also never distributed; 1076 was built from a different branch that MISSED the handler-deadline fix). (c) Client-side partial-failure Error→Empty conversion fixed in 1cbf364c (landed post-1076, never distributed).
Fix landed?	Partial — 1cbf364c (client-side Error→Empty) landed but NOT distributed. Engine handler-deadline fix not in 166ef2e7 (current HEAD is pre-fix).
Covering device Challenge FQN	Challenge58SearchReturnsResults (new). Must drive: onboard one provider → navigate to search → enter query → submit → assert at least one result row is displayed (user-visible, not an Error or Empty screen).
Reproduce-first protocol	Mutation: in SearchResultViewModel, make streamSearch always emit SearchResultUiState.Error("Something went wrong") immediately after submission, bypassing the real API call. RED run: Challenge58 must fail with assertion like AssertionError: No SearchResultItem nodes found; expected at least 1 result. GREEN run: after reverting the mutation, the Challenge must PASS against the real engine.
LVA-008 blocker	YES — Challenge58 navigates to search_result which is a nested route. If LVA-008 nav-teardown fires, the test crashes with IllegalStateException: State must be at least 'CREATED'.

2.2 LVA-084 (Video #2) — Onboarded provider NOT used by Search; unconfigured providers active as filters

Field	Value
ID	LVA-084
Title	Video #2 — Onboarded provider (YTS) is NOT the provider set used by Search; unconfigured providers active as filters
Status	In progress
Priority	P0
Root cause	SearchInputViewModel.kt:45 initialized selectedProviders = availableProviders (ALL providers, not just onboarded ones). Fix landed in

Field	Value
	8c795d22: keepOnlySearchEnabled reconciler that filters search providers to the onboarded+enabled set.
Fix landed?	YES — 8c795d22 committed but NOT distributed (post-1076).
Covering device Challenge FQN	Challenge59SearchUsesOnboardedProviders (new). Must: onboard exactly one provider (e.g. YTS) → navigate to search → tap the provider filter chip → assert ONLY the onboarded provider appears as a selected/available chip. Assert the total chip count equals the onboarded count.
LVA-008 blocker	Partial — the search-input route (search/search_input) is also nested. If LVA-008 manifests in search_input, the chip assertions cannot even execute.

2.3 LVA-079 (Video #3) — Search-input chips vs results-filter chips disagree

Field	Value
ID	LVA-079
Title	Video #3 — search-input chips vs results-filter chips disagree + results chip set non-deterministic run-to-run
Status	In progress
Priority	P1
Root cause	The input chips (what you see in SearchInputScreen) and the results filter chips (what renders in SearchResultScreen) are sourced from DIFFERENT repositories. The input chips come from SearchInputViewModel.selectedProviders (populated from ProviderConfigRepository.observeSelected() or its legacy path). The results chips come from SearchResultViewModel's own provider-tracking (populated from the multiSearch SSE response's provider_done/provider_error events, NOT from the input selection). Non-determinism: the SSE response order depends on network latency; providers return in different order per run, creating a different chip order in results vs input.
Fix scope remaining	Add a ResultsProviderFilterChipMapper (or extend SearchResultViewModel) that sources the chip list from

Field	Value
	the SAME selectedProviders that the input chips use, not from the SSE event order. Store the intended provider order at submit-time so the results chip order is stable and matches input.
Covering device Challenge FQN	Challenge60InputResultsChipsAgree (new). Must: onboard at least 2 providers → navigate to search → submit a query → wait for results → capture the list of provider chip labels from the input screen AND the results screen → assert they match (same set, same order).
Flakiness guard	THE INPUT CHIPS MAY CHANGE between navigation and submit. The Challenge must snapshot input chips at submit-time (not at navigation-time). The assertion is: input chips at submit-time == results chips at first-results-time.

2.4 LVA-085 (Video #4) — Provider id labels shown raw/lowercased in results filter chips

Field	Value
ID	LVA-085
Title	Video #4 — Provider id labels shown raw/lowercased ('torrentdownloads','archiveorg','kinozal','yts') in results filter chips
Status	In progress (current HEAD 166ef2e7)
Priority	P1
Root cause	Results filter chips render <code>provider.id</code> (raw lowercase id, e.g. "yts", "torrentdownloads") instead of <code>provider.displayName</code> (e.g. "YTS", "Torrent Downloads"). The commit 166ef2e7 tracks LVA-085 in progress — a fix that sorts the provider IDs exists on the input-chip side but does NOT address the RESULTS-chip display-name mapping. Per commit body: the LVA-079 "fix" was itself a §6.AB bluff (sorted the raw id LIST; its JVM test asserted the id list, never the rendered LABEL).
Fix scope	Add <code>displayName</code> field to the chip-rendering data class in <code>SearchResultViewModel</code> (or wherever results chips are built). Wire it to <code>ProviderInfo.displayName</code> which already exists on the SDK-provided descriptor. The raw

Field	Value
	id should ONLY be used as an internal key, never as displayed text.
Covering device Challenge FQN	Challenge61ResultsChipsShowDisplayNames (new). Must: onboard multiple providers that have different id vs displayName (rutracker→"RuTracker", kinozal→"Kinozal", archiveorg→"Internet Archive") → search → capture the results filter chip labels → assert each label equals the provider's displayName (not its id).
Reproduce-first mutation	Mutation: in the chip-rendering code, force text = provider.id instead of provider.displayName. RED run: Challenge61 fails with AssertionError: expected "RuTracker" but found "rutracker".

2.5 LVA-086 (Video #5) — No empty-state and no loading indicator on search results

Field	Value
ID	LVA-086
Title	Video #5 — No empty-state and no loading indicator on search results (perceived hang)
Status	In progress
Priority	P1
Root cause	SearchResultScreen renders CircularProgressIndicator only during the initial loading phase. Once the search returns ZERO results (empty list), it renders nothing — a blank screen. There is no EmptyState composable, no "No results" text, no retry button. Since there is no empty-state and no loading indicator (and the search may appear to hang because there is no visible feedback), the user perceives the app is stuck.
Fix scope	Add Loading and Empty sealed variants to the SearchResultUiState. The Loading variant renders a CircularProgressIndicator + "Searching..." text (or a shimmer placeholders). The Empty variant renders an EmptyState composable ("No results found. Try a different query."). The Error variant already exists (from 1cbf364c) — verify it includes a retry button.

Field	Value
Covering device Challenge FQN	Challenge62SearchEmptyState (new). Must: onboard a provider → search for a nonsense query that is guaranteed to return zero results (e.g. "zxzxzxzxzxzxzxzx" on YTS) → wait for the search to complete → assert an EmptyState composable is displayed ("No results" text visible).
Reproduce-first mutation	Mutation: in SearchResultViewModel, when the result list is empty, set state to Idle instead of Empty. RED run: Challenge62 fails with AssertionError: No "No results" text found.

2.6 LVA-087 (Video #6) — Welcome claims '4 providers available' but picker lists ~12

Field	Value
ID	LVA-087
Title	Video #6 — Welcome claims '4 providers available' but picker lists ~12
Status	In progress
Priority	P2
Root cause	The Welcome screen's header text has a hardcoded or stale count: "4 providers available" (or similar). This was accurate when only 4 built-in providers existed (rutracker, rutor, kinozal, nnmclub) but has not been updated to reflect the now ~12 available providers (5 built-in + 8 curated on-device + possible others).
Fix scope	Replace the hardcoded count with a live query: <code>providerRegistry.all().count { it.supported }</code> or equivalent, reading from the ProviderCatalogRepository at Welcome-screen load time. The count should update dynamically if providers are added/removed. Per §6.R, no hardcoded integer literal for the count.
Covering device Challenge FQN	Challenge63WelcomeCountMatchesPicker (new). Must: navigate to the Welcome screen → read the header text that contains the provider count ("N providers available" or "N providers supported") → tap "Get Started" → on the Provider Picker screen, count the

Field	Value
	visible provider entries → assert the Welcome count matches the picker count.
Reproduce-first mutation	Mutation: hardcode the Welcome count to "4 providers available". RED run: Challenge63 fails with AssertionError: expected "12" but found "4" (or similar mismatch).

2.7 LVA-088 (Video #7) — 'Choose your API' shows 'lava.app:7777' preset + mislabeled 'On this network'

Field	Value
ID	LVA-088
Title	Video #7 — 'Choose your API' shows 'lava.app:7777' preset + mislabeled 'On this network'
Status	In progress
Priority	P2
Root cause	The API discovery screen (introduced via commit in the 60th §6.L cycle as OnboardingStep.ApiSelection) uses a preset text field that shows "lava.app:7777" as a placeholder. This is a legacy port from the old manual-entry API endpoint config. The mDNS-discovered APIs are labeled "On this network" rather than showing the discovered service name (e.g., "Lava API on My Desktop"). The port 7777 does not match any running service (the on-device api-app uses 8543 dev / 8443 prod; the proxy uses 8080). Remove the "lava.app:7777" hard-coded preset placeholder. For mDNS-discovered APIs, use the service's advertised name
Fix scope	(NsdServiceInfo.getServiceName()) as the display label instead of the generic "On this network" string. Preserve the manual-entry fallback for advanced users but with a better label ("Enter URL manually").
Covering device Challenge FQN	Challenge64ApiDiscoveryScreenFriendlyNames (new). Must: navigate to the API selection screen → observe all listed APIs → assert no entry shows the raw string "7777" or "lava.app" → assert each mDNS-discovered entry shows a non-generic name (not "On this network").

Field	Value
Reproduce-first mutation	Mutation: keep the "lava.app:7777" preset in the API selection screen. RED run: Challenge64 fails with AssertionError: Found unexpected text "lava.app:7777".

2.8 LVA-089 (Video #8) — mDNS-discovered API shows raw IP with no friendly name

Field	Value
ID	LVA-089
Title	Video #8 — mDNS-discovered API shows raw IP 192.168.0.107:8443 with no friendly name
Status	In progress
Priority	P2
Root cause	The mDNS discovery flow (LocalNetworkDiscoveryService + DiscoverLocalEndpointsUseCase) returns discovered services with their host.hostAddress (raw IP) as the primary display field. The Endpoint sealed interface has a name property that is not populated from mDNS service attributes. The Friendly Name attribute (_lava-api._tcp.local.'s TXT record) is either not being parsed or not propagated to the UI. In DiscoverLocalEndpointsUseCase (or the downstream mapper at core/data/src/.../la/NetworkDiscoveryMapper.kt if it exists), populate Endpoint.name from the NsdServiceInfo attributes (TXT record key "name"). Fallback: if TXT record is absent, use the service type + hostname ("lava-api at hostname.local"). NEVER display the raw IP as the primary name. Per §6.R: IP strings must not be hardcoded in display templates; use the resolved name.
Fix scope	
Covering device Challenge FQN	Challenge65MdnsShowsFriendlyName (new). Must: trigger mDNS discovery → when an API is discovered → assert the displayed name is NOT a raw IP address (does not match regex <code>^\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}:\\d+\\\$</code>) → assert the name is a human-readable string ((>)3 chars, no colon).

Field	Value
Reproduce-first mutation	Mutation: in <code>DiscoverLocalEndpointsUseCase</code> , force the endpoint name to <code>host.hostAddress.toString()</code> (raw IP). RED run: Challenge65 fails because the displayed name matches the raw-IP regex.

2.9 LVA-090 (Video #9) — 'Select all' silently enables auth-requiring providers

Field	Value
ID	LVA-090
Title	Video #9 — Onboarding 'Select all' silently enables auth-requiring (Captcha/Form Login) providers
Status	In progress
Priority	P3
Root cause	The "Select all" button in the Provider Picker toggles EVERY provider regardless of its <code>AuthType</code> — including <code>FORM_LOGIN</code> (rutracker, nnmclub) and <code>CAPTCHA</code> (kinozal) providers that require credentials. The selected set is used as the search filter set; auth-requiring providers that have NOT been configured with credentials cause search failures or errors downstream.
Fix scope	When "Select all" is clicked, filter the selection to only include providers where <code>authType == NONE</code> or where the user has already configured credentials. Add a warning subtitle when "Select all" is active: "Some providers requiring credentials were not selected automatically." Or disable "Select all" entirely and replace with a "Select all anonymous" option.
Covering device Challenge FQN	Challenge66SelectAllDoesNotEnableAuthProviders (new). Must: navigate to the Provider Picker → tap "Select all" → assert that NO provider with <code>AuthType == FORM_LOGIN</code> or <code>AuthType == CAPTCHA</code> is selected (checked). Assert that all <code>AuthType == NONE</code> providers are selected.
Reproduce-first mutation	Mutation: make "Select all" select every provider unconditionally, including auth-requiring ones. RED run: Challenge66 fails because a <code>FORM_LOGIN</code> provider is found in the selected state.

2.10 LVA-091 (Video #10) — App-ID co-mingling (UNCONFIRMED)

Field	Value
ID	LVA-091
Title	Video #10 — App-ID co-mingling (debug .dev + release both labeled 'Lava') — UNCONFIRMED in video
Status	Queued
Priority	P3
Root cause	UNCONFIRMED. The operator noted in the video that both the debug APK (applicationId <code>digital.vasic.lava.client.dev</code>) and the release APK (<code>digital.vasic.lava.client</code>) show the same display name "Lava" on the home screen. This may cause confusion when both are installed but is the expected Android behavior — application label from the manifest, not the app ID.
Actions before closing	(1) Confirm whether the two APKs actually install side-by-side on the S23 Ultra (test on real device). If they do, the "co-mingling" is cosmetic and expected. (2) If they don't install side-by-side, investigate <code>android:label</code> override logic or <code>installLocation</code> conflicts.
Covering device Challenge FQN	None needed if confirmed cosmetic. If a real conflict is found, Challenge67AppIdSeparation (new) must assert both APKs install on the same device without conflict.
Priority override	If confirmed NOT cosmetic, escalate to P1.

3. Coverage-Intersection Gate Mapping

This section defines how the mechanical `scripts/check-cycle-coverage.sh` (owed via \$6.AK-debt) will map CHANGELOG claims to `executed+passed` device Challenge rows.

3.1 Claim-to-Challenge mapping mechanism

Each cycle's author (or the agent drafting the CHANGELOG) maintains a **cycle-coverage-map** — a lightweight key-value mapping that lives as a YAML block at the top of the CHANGELOG entry for that version, or as a standalone file at `.lava-ci-evidence/<version>/cycle-coverage-map.yaml`. Format:

```
version: "1.3.13-1077"
claims:
  - bullet: "Search now returns real results (no more 'Something
    went wrong')"
    covers_issues: ["LVA-083"]
    covering_challenge: "Challenge58SearchReturnsResults"
  - bullet: "Search filters follow the providers you onboarded"
    covers_issues: ["LVA-084"]
    covering_challenge: "Challenge59SearchUsesOnboardedProviders"
  - bullet: "Provider chips show friendly names (RuTracker,
    Internet Archive) instead of raw ids"
    covers_issues: ["LVA-085"]
    covering_challenge: "Challenge61ResultsChipsShowDisplayNames"
```

3.2 Gate execution flow

1. `scripts/firebase-distribute.sh` (Phase 1 Gate 6, or the new \$6.AK Phase-1 Gate) loads the `cycle-coverage-map` for the CURRENT version.
2. For each `covering_challenge` entry, the gate parses the \$6.Z evidence file at `.lava-ci-evidence/distribute-changelog/<channel>/<version>-<code>-test-evidence.{md,json}`.
3. It asserts that the evidence file contains an EXECUTED row for that Challenge FQN with a PASS verdict, that the row's commit SHA matches the working-tree HEAD, and that the row's timestamp is within 24h.
4. If ANY covering Challenge is missing / failed / stale-SHA / stale-timestamp, the gate rejects with a directive naming the uncovered claim.
5. On rejection, the distribute entry point writes a FATAL-level message to stderr and exits 1. It does NOT offer a bypass flag.

3.3 Three-claimant gate (minimum mandatory)

Even if a cycle has ZERO CHANGELOG claims (e.g. a hotfix for a single crash with no feature changes), the cold-start C00 Challenge is mandatory per §6.Z. The §6.AK gate therefore has a **three-claimant minimum**:

1. **C00** — Cold-start crash survival (always mandatory per §6.G / §6.Z / §6.AK).
2. **C01** — App launch + tracker selection (always mandatory per §6.G).
3. **The covering Challenge(s) for every user-visible fix in the CHANGELOG** (zero or more).

A distribute with C00 alone is ALWAYS a §6.AK violation unless the CHANGELOG is also empty.

4. Reproduce-First Protocol Template

This template MUST be followed for every covering device Challenge written to meet §6.AK clause 2. Each Challenge file's KDoc MUST include a **FALSIFIABILITY REHEARSAL** block following this template.

4.1 Template

```
/**
 * [Challenge name] – [one-line description of what user-visible
 *   behavior this verifies]
 *
 * ## §6.AK Reproduce-First Protocol
 *
 * ### RED run (before fix)
 * 1. Apply the mutation: [precise description of what to change
 *   in which production file(s)].
 * 2. Build the androidTest APK via `./
 *   gradlew :app:assembleDebugAndroidTest`.
 * 3. Install and run THIS Challenge only on the device:
 *   `adb shell am instrument -w -e class lava.app.challenges.
 *   [ChallengeName] digital.vasic.lava.client.dev.test/
 *   androidx.test.runner.AndroidJUnitRunner`
 * 4. Expected failure: [copy-paste of the assertion failure
 *   message the test produces].
 *
 * ### GREEN run (after fix)
 * 5. Revert the mutation (git checkout the production file(s)).
 * 6. Rebuild and re-run the identical Challenge.
 * 7. Expected pass: [description of the user-visible state the
 *   test asserts].
```

```

*
* ### Evidence file reference
* The RED and GREEN outputs are recorded at:
* `.lava-ci-evidence/reproduce-first/[YYYY-MM-DD]/
*   [ChallengeName]-RED.txt`
* `.lava-ci-evidence/reproduce-first/[YYYY-MM-DD]/
*   [ChallengeName]-GREEN.txt`
*
* ### Mutation type
* [NON-CRASHING BREAK – the mutation causes incorrect user-
*   visible behavior,
*   not a crash, per §6.AB.3]
* OR
* [CRASH REPRODUCTION – the mutation causes the exact Crashlytics
*   stack trace
*   from issue [issue-ID], per §6.AK clause 3]
*
* ### LVA-008 dependency
* This Challenge navigates to [route], which is a nested route.
*   The §6.AK gate
* requires this test to execute on-device; see docs/issues/
*   2026-06-26-nav-teardown.md
* for LVA-008 bypass/workaround status.
*/

```

4.2 Example filled-in template for Challenge61

```

/**
* Challenge61 – Results filter chips show display names (not raw
*   ids)
*
* ## §6.AK Reproduce-First Protocol
*
* ### RED run (before fix)
* 1. In SearchResultViewModel.kt, at the point where results chip
*   labels are set,
*   change `chipLabel = provider.displayName` to `chipLabel =
*   provider.id`.
* 2. Build androidTest APK.
* 3. Run: `adb shell am instrument -w -e class
*   lava.app.challenges.Challenge61 ...`
* 4. Expected failure: `AssertionError: expected "RuTracker" but
*   found "rutracker"`
*   (or for any displayName-id mismatch provider).
*
* ### GREEN run (after fix)
* 5. Revert the mutation (git checkout SearchResultViewModel.kt).
* 6. Rebuild and re-run.
* 7. Expected pass: all chip labels equal the provider's
*   displayName field, and
*   NOT equal the raw id.
*
* ### Mutation type

```

```
* NON-CRASHING BREAK – the mutation causes incorrect user-visible  
text but no crash.  
*/
```

5. §6.AK-debt Spec

§6.AK-debt (commit 627a0d58) names the mechanical enforcement tooling as OWED. This section defines it concretely.

5.1 scripts/check-cycle-coverage.sh

A new script at scripts/check-cycle-coverage.sh with the following contract:

Purpose: Verify that every CHANGELOG-claimed user-visible fix for the current version has an EXECUTED+PASSED covering device Challenge in the matching §6.Z evidence file.

Arguments:

```
--version <versionName-versionCode>  # e.g. "1.3.13-1077"  
(default: auto-detect from app/build.gradle.kts)  
--channel <channel>                    # "debug" or "release"  
(default: "debug")  
--evidence-dir <path>                  # path to the evidence  
directory (default: auto-resolve from distribute-changelog)  
--map <path>                           # path to the cycle-  
coverage-map YAML (default: auto-resolve from evidence dir)  
--strict                               # exit 1 on ANY uncovered  
claim (default: --strict in gate mode)
```

Exit codes:

- 0 — All claims covered by executed+PASSED device Challenges.
- 1 — One or more claims lack a covering executed+PASSED Challenge. Stderr names each uncovered claim.
- 2 — Evidence file missing, cycle-coverage-map missing, or version/commit-SHA mismatch.
- 3 — Internal error (malformed YAML, evidence parsing failure).

Steps:

1. Resolve version, channel, evidence dir, and cycle-coverage-map path.
2. Read the cycle-coverage-map YAML: extract `claims[].covering_challenge` for each claim whose `covers_issues`

intersects the cycle's work (or ALL claims if no explicit issue mapping).

3. Read the §6.Z evidence file for the matching version+channel+commit-SHA.
4. For each covering_challenge, assert:
 - The evidence file contains a row for that Challenge FQN.
 - That row's verdict is PASS (not FAIL, not SKIP).
 - That row's device is a cold-booted emulator inside a container or a real device (runner != "host-direct").
 - That row's timestamp is within 24h of the current time.
 - That row's commit SHA equals the working-tree HEAD.
5. If any assertion fails, exit 1 (or 2 for structural issues). If all pass, exit 0.

5.2 Hermetic test fixtures

Under tests/cycle-coverage/:

Fixture	Description	Expected exit
positive_all_covered/	CHANGELOG claims 2 fixes; cycle-coverage-map lists 2 Challenges; evidence file has PASS rows for both.	0
negative_claim_missing_challenge/	CHANGELOG claims 1 fix; cycle-coverage-map lists 0 covering Challenges (or the map entry is absent).	1
negative_challenge_compiled_not_executed/	Cycle-coverage-map lists 1	1

Fixture	Description	Expected exit
	Challenge; evidence file has its FQN but the verdict is SKIP (or row has no PASS).	
negative_stale_evidence/	Evidence file PASS rows exist but timestamp > 24h from test time.	1
negative_wrong_sha/	Evidence file PASS rows exist but commit SHA does not match working-tree HEAD.	2

5.3 Phase-1 Gate in scripts/firebase-distribute.sh

A new Phase 1 Gate (numbered Gate 7, following the existing §6.Z Gate 6, or replacing Gate 6 by composing both) MUST be added to scripts/firebase-distribute.sh:

```
# Phase 1 – Gate 7: §6.AK cycle-coverage check
info "Phase 1 – Gate 7: §6.AK cycle-coverage (claims × executed
device Challenges)"
check-cycle-coverage.sh --version="${VERSION}" --channel="$
{CHANNEL}" --strict
case $? in
  0) info "§6.AK gate PASS – all CHANGELOG claims covered by
executed device Challenges" ;;
  1) fatal "§6.AK gate FAIL – these CHANGELOG claims have no
covering executed device Challenge" ;;
  2) fatal "§6.AK gate FAIL – evidence or mapping file missing/
stale" ;;
  3) fatal "§6.AK gate FAIL – internal error in cycle-coverage
```

```
scanner" ;;  
esac
```

5.4 Pre-push hook integration

.githubhooks/pre-push MUST grow a Check 10 (following §6.Z-debt's Check 9 pattern) that detects when the pushed commit would advance a version pointer in .lava-ci-evidence/distribute-changelog/<channel>/last-version. On detection, the hook runs scripts/check-cycle-coverage.sh --strict and rejects if it exits non-zero. This prevents pushing a version-advance (which signifies "ready to distribute") without the covering device evidence.

6. Search Root-Cause Continuation Plan

6.1 Current state (as of 166ef2e7, 2026-06-26)

The investigation from docs/issues/2026-06-24-search-timeout-and-interrupt-rootcause.md and docs/issues/2026-06-24-search-timeout-coordination-analysis.md has identified:

KNOWN ROOT CAUSES (proven by evidence):

1. **H1 — 401 auth-header overwrite** (AuthInterceptor at NetworkModule.kt overwrites the engine's API key with an empty or expired token before the search request reaches the server). Fix shipped in 1072. VERIFIED FIXED by the Crashlytics absence of 401-related SocketTimeout after 1.3.11-1073. **§6.AK status:** Challenge58 covers this path — if auth overwrite recurs, search returns Error, not results.
2. **H2 — Engine handler has NO deadline** (search.go:64 — p.Search(c.Request.Context(), ...) with no context.WithTimeout). GetSearch handler does not cap the search call, so a slow provider (YTS with 4 mirrors × 8s failover) can consume up to 32s, which exceeds the Android client's 30s readTimeout. Fix (the 18s handler deadline) landed in 1073 but 1073 was never distributed to testers. **1076 was built from a branch that MISSED this fix. §6.AK status:** The engine-deadline fix MUST be in the next distribute. The lava-api-go handler test (TestGetSearchHandler_Deadline) must prove that GET /v1/yts/search returns within 20s even when YTS mirrors are all unreachable.

3. H3 — Client-side partial-failure Error→Empty conversion

(1cbf364c). When a provider fails (timeout, 401, DNS error), `SearchResultViewModel.handleStreamEnd()` was converting partial failures into `SearchResultUiState.Empty` instead of `SearchResultUiState.Error`. This made "zero results" indistinguishable from "search failed." Fix landed in 1cbf364c but NOT distributed. **§6.AK status:** The Error→Empty fix MUST be in the next distribute. Challenge58 covers this by asserting the search produces RESULTS (not Empty) when providers are healthy.

4. H4 — Back-press during search does not cancel —

`SearchResultViewModel.onCreate` fires `observeStreamMultiSearch` which blocks on `sdk.streamMultiSearch(...).collect{}` for up to 30s. No `onBackClick` cancellation exists. Fix described in the root-cause doc (track the job and cancel on `BackClick`) but NOT YET IMPLEMENTED in any commit. **§6.AK status:** Fix OWED in this cycle.

6.2 Remaining unknowns

Unknown	Evidence gap	Next step
Which specific provider(s) caused the 1076 "Something went wrong"? 1076 has the 401 auth fix (H1 shipped) but NOT the engine deadline (H2) nor the Error→Empty fix (H3).	Operator must provide adb logcat from a 1076 debug APK, or Crashlytics non-fatals from the 1076 release. Without these, the exact failure path is UNKNOWN.	Request operator to reinstall 1076 debug APK, reproduce the search failure, and share <code>adb logcat -b main -e lava</code> . Or wait for Crashlytics to receive events from the 1076 release (already distributed).
Is YTS the only provider triggering the timeout, or do other curated providers also contribute?	The Crashlytics sample from 1072 shows <code>provider=yts</code> . The coordination analysis shows YTS is the most likely due to 4-mirror failover, but any slow provider can trigger it.	Once the engine deadline (H2) lands, even ALL providers slow would produce fast errors, not 30s hangs. Monitor Crashlytics after the next distribute for <code>error=timeout</code> with non-yts providers.

6.3 Next debugging steps (if search still fails after H2+H3+H4 land)

If, after landing the engine handler deadline (H2), the Error→Empty fix (H3), and the back-press cancellation (H4), the operator reports "search still returns no results on my device," the following diagnostic chain applies:

1. Check Crashlytics non-fatals — the

`SearchResultViewModel.recordProviderFailure` call (shipped in 1071/1072) writes per-provider failure diagnostics. If the non-fatals show timeout for YTS but the engine has the 18s deadline, the issue is likely in the on-device network path (DNS blocking, TLS interception, mobile carrier proxy). If non-fatals show 401 or 403, the auth-key exchange flow is still broken.

2. Check adb logcat from the device — key markers to grep for:

- `lava_search_submit` — the search was initiated
- `lava_search_result_count` — the number of results returned (0 = empty, N = results)
- `lava_provider_error` — per-provider failure with cause
- `lava_api_disconnected` — engine not reachable at all

3. If Crashlytics is silent (no non-fatals logged), the engine may not be reachable from the device. Check mDNS discovery (`lava_mdns_resolved` log marker) and the API connectivity probe.

4. If Crashlytics shows engine errors but client shows Empty, verify the 1cbf364c fix is present (client treats partial-failure as Error not Empty). If the fix IS present but Empty still appears, there is a SECOND path in the state machine that sets Empty instead of Error — investigate `handleStreamEnd()` in `SearchResultViewModel.kt` for all exit points.

6.4 Engine-side fix checklist (lava-api-go)

From the coordination analysis (§3 recommended values):

Change	File	Current value	Target value	Status
Add <code>context.WithTimeout</code> to <code>GetSearch</code> handler	<code>lava-api-go/internal/handlers/v1/search.go:64</code>	NONE (no deadline)	20 * <code>time.Second</code>	OWED
	<code>lava-api-go/internal/</code>	30 * <code>time.Second</code>	20 * <code>time.Second</code>	OWED (align

Change	File	Current value	Target value	Status
Reduce GetMultiSearch per-provider deadline	handlers/v1/search.go:201			with single-provider)
Reduce YTS DefaultTimeout	lava-api-go/internal/provider/curated/yts/client.go:46	20s	15s	OWED
Reduce YTS perAttemptTimeout	lava-api-go/internal/provider/curated/yts/client.go:51	8s	5s	OWED
Reduce TPB/TorrentsCSV perAttemptTimeout	respective client.go	8s	5s	OWED
Go server WriteTimeout	lava-api-go/internal/server/server.go	NONE	25s	OWED
Engine-side coverage test (handler deadline)	lava-api-go/internal/handlers/v1/search_test.go	N/A	NEW test: slow provider → handler returns by deadline	OWED

6.5 Client-side fix checklist

Change	File	Current behavior	Target behavior
Cancel in-flight search on BackClick	SearchResultViewModel.kt	No cancellation	Track the streamMultiSe job; cancel on BackClick side effect
Raise LAN readTimeout	core/network/impl/.../NetworkModule.kt:174	30s	45s
	core/network/impl/.../NetworkModule.kt	NONE	

Change	File	Current behavior	Target behavior
Add LAN callTimeout			120s (SSE mult search safety backstop)
Client-side coverage test (back- press cancellation)	feature/search_result/src/test/.../ SearchResultViewModelTest.kt	N/A	NEW test: slow search → BackClick → search cancell
Challenge58 (search returns results)	app/src/androidTest/.../ Challenge58SearchReturnsResultsTest.kt	N/A	NEW device Challenge

7. Implementation Phasing

Phase 1: P0 fixes + covering Challenges + LVA-008 workaround

Gate: device Challenges that prove search+filtering works end-to-end.

Item	Workable ID	Depends on	Estimated effort
Engine handler deadline (18s/20s) + per-attempt timeout reduction	LVA-083 (H2)	None (standalone Go change)	1 session
Go handler deadline test	LVA-083	Engine handler deadline change	Same commit
Client Error→Empty fix (already landed in 1cbf364c)	LVA-083 (H3)	None — merge 1cbf364c into the distribute branch	Git-merge only (if not already in branch)
Client back-press cancellation	LVA-083 (H4)	None (standalone ViewModel change)	1 session

Item	Workable ID	Depends on	Estimated effort
Client search-cancellation ViewModel test	LVA-083 (H4)	Back-press cancellation change	Same commit
Search-filter reconcile (already landed in 8c795d22)	LVA-084	None — merge 8c795d22 into distribute branch	Git-merge only
Challenge58 — search returns results (device)	LVA-083	LVA-008 workaround for nested-route navigation	1-2 sessions (depends on LVA-008 status)
Challenge59 — search uses onboarded providers (device)	LVA-084	LVA-008 workaround	1 session

LVA-008 dependency for Phase 1: Both Challenge58 and Challenge59 navigate to nested routes (`search_result` and `search_input` respectively). If LVA-008 blocks all nested-route Challenges, Phase 1 cannot produce EXECUTED device attestations. Three options for the §6.AK gate:

1. **Fix LVA-008 first** — the upstream navigation-defect workaround (Activity-scoped `LocalLifecycleOwner` or equivalent) in same cycle. This is the cleanest option but has been falsified 6+ times.
2. **Workaround LVA-008 in the Challenge itself** — e.g., capture the search result screen content via `ComposeTestRule` before navigating back (avoiding the teardown race). Investigate if the crash only occurs on back-navigation, not on forward-navigation to the result screen.
3. **Strike LVA-083 "search fixed" from CHANGELOG** if device Challenges cannot execute due to LVA-008. This is the §6.AK-clause-6 honest option: "we believe the fix is correct but we cannot prove it on device because LVA-008 blocks all nested-route tests." The `lava-api-go` engine-side Go tests serve as partial unit evidence, but §6.AK clause 2 requires ON-DEVICE reproduction for UI/flow issues; the CHANGELOG would reflect SEARCH-FIX-PENDING (partial, unverified on device) rather than "search works again."

Phase 2: P1 fixes + covering Challenges

Gate: device Challenges that prove chip agreement, display-name rendering, and empty-state work.

Item	Workable ID	Depends on	Estimated effort
Results chip provider-order stability (source from same repo as input chips)	LVA-079	None (standalone ViewModel change)	1 session
JVM test for chip-order agreement	LVA-079	Chip-order fix	Same commit
Results chip display-name mapping	LVA-085	None (standalone mapping change)	0.5 session
JVM test for display-name rendering in chips	LVA-085	Display-name fix	Same commit
Challenge60 — input vs results chips agree (device)	LVA-079	LVA-008 workaround for nested routes	1 session
Challenge61 — results chips show display names (device)	LVA-085	LVA-008 workaround for nested routes	1 session
Challenge62 — search empty-state shown (device)	LVA-086	LVA-008 workaround	1 session
Loading indicator + EmptyState composable	LVA-086	None (standalone UI change)	1 session
ViewModel test for Empty state	LVA-086	EmptyState composable	Same commit

Phase 3: P2-P3 fixes + covering Challenges

Gate: device Challenges that prove Welcome count, API discovery labels, mDNS names, and Select-All behavior are correct.

Item	Workable ID	Depends on	Estimated effort
Dynamic Welcome provider count	LVA-087	None (standalone text change)	0.5 session
Remove hardcoded "lava.app:7777" preset	LVA-088	None (standalone UI text change)	0.5 session
mDNS friendly-name propagation	LVA-089	None (standalone UseCase/mapper change)	1 session
"Select all" skips auth-requiring providers	LVA-090	None (standalone picker filter logic)	1 session
LVA-091 confirmation (cosmetic vs real conflict)	LVA-091	Device test to install both APKs	Investigation only
Challenge63 — Welcome count matches picker (device)	LVA-087	None (Welcome + Picker are non-nested routes)	0.5 session
Challenge64 — API discovery labels (device)	LVA-088	None (API selection is early onboarding, non-nested)	0.5 session
Challenge65 — mDNS shows friendly name (device)	LVA-089	LVA-089 fix	1 session
Challenge66 — Select All does not enable auth providers (device)	LVA-090	LVA-090 fix	0.5 session
Challenge67 — App-ID separation (device, only if confirmed needed)	LVA-091	LVA-091 confirmation	0.5 session

8. CHANGELOG Honesty Contract for the Next Distribute

Per §6.AK clause 6, the CHANGELOG for the next distribute entry (target version after 166ef2e7) MUST honestly reflect what the device gate actually proves. The template below shows the required structure. Any bullet whose covering Challenge did NOT execute and PASS on the gate is STRICKEN from the CHANGELOG before distribute.

Lava-Android-1.3.13-1077 – 2026-06-26 (Phase 1: search+filtering + device gate)

Previous published: Lava-Android-1.3.12-1076 (§6.AK incident – C00-only gate).

1077 is the FIRST distribute that meets the §6.AK gate: every CHANGELOG claim has an EXECUTED+PASSED covering device Challenge. LVA-008 workaround: [describe method].

- **Search actually returns results.** The engine now has an 18s handler deadline (no more SocketTimeout), the partial-failure Error→Empty bug is fixed, and back during a search cancels immediately. Verified by Challenge58 on device.
[§6.AK reproduce-first RED→GREEN recorded]
- **Search filters match your onboarded providers.** Unconfigured providers no longer appear as search filters. Verified by Challenge59 on device.
- **[P1 claims only if Phase 2 also landed: ...]**
- **[P2-P3 claims only if Phase 3 also landed: ...]**

Known open (not claimed as fixed):

- LVA-008 nav-teardown: still present on nested routes; device Challenges in this cycle worked around it via [method]. Upstream minimal-repro authored.
- LVA-079 chip agreement: P1, Phase 2 target.
- LVA-085 display names: P1, Phase 2 target.
- LVA-086 loading/empty state: P1, Phase 2 target.
- LVA-087..LVA-091: P2-P3, Phase 3 targets.

Device gate evidence:

.lava-ci-evidence/distribute-changelog/debug/1.3.13-1077-test-evidence.json

9. Honest Scope Statement

"This cycle delivers a functioning search+filter experience (P0 video issues #1 and #2) with the first-ever mechanically enforced §6.AK device gate that proves each CHANGELOG claim on a real device. P1 and P2-P3 issues (chip labels, empty state, Welcome copy, API discovery) are targeted for phases 2 and 3 respectively. The LVA-008 nav-teardown crash remains open; Phase 1 device Challenges work around it by [determined at implementation time]. The §6.AK-debt mechanical gate (`scripts/check-cycle-coverage.sh`) ships as a hermetic-test-proven scanner but the Phase-1 hook integration is OWED-until-implemented per the existing debt pattern."

10. Constitutional Notes

- **§6.AK:** This spec IS the implementation plan for §6.AK clauses 1-7 across the 10 open video issues. Every section above maps directly to one or more §6.AK clauses.
- **§6.AD.8 (HelixConstitution inheritance):** The §6.AK clause is already present in `root CLAUDE.md` (added by commit 627a0d58). This spec's per-issue coverage-maps and reproduce-first templates ARE the per-scope mechanical implementation. The §6.AK-propagation check (CM-COVENANT-114-AK-PROPAGATION) is owed via §6.AK-debt as part of the cycle-coverage scanner.
- **§6.Z:** The §6.Z evidence file (per-distribute test-execution record) is the SOLE evidence source for §6.AK. No parallel evidence track. The Phase 1 Gate in §6.AK-debt reads the same file that §6.Z Gate 6 reads — they compose.
- **§6.AA (two-stage distribute):** The §6.AK gate applies to BOTH stages. Debug stage 1 must have covering Challenges executed and passed. Release stage 2 must have the SAME covering Challenges executed and passed against the release APK (not just the debug APK). The cycle-coverage-map is shared; the evidence file is per-channel.
- **§6.AB (discrimination):** Every covering Challenge in this spec has a documented Reproduce-first mutation (the NON-CRASHING BREAK or CRASH REPRODUCTION) per §6.AB.3. The Challenge is NOT complete until the mutation → RED → revert → GREEN cycle is recorded.
- **§6.R:** No hardcoded provider counts, port numbers, or IP addresses in the fix code. All such values must come from the provider registry, config, or mDNS discovery — never as string/int literals.

- **§6.AC:** All non-fatal error paths (search timeout, provider error, back-press-cancellation) must call `analytics.recordNonFatal` with `{feature: "search", operation, provider, screen: "search_result"}`. The Crashlytics non-fatal channel IS the §6.AK evidence for "crash was reproduced on device."
- **§6.Y (post-distribute version bump):** After this cycle's distribute, the version MUST be bumped before any new code changes land. The next version's CHANGELOG entry must start with an empty-fill or the follow-up cycle's work.